

1. Tematica del proyecto

El proyecto simula cómo estructurar el código de una infraestructura web real y está compuesto por tres ficheros:

- Desplegable (Interfaz): Obliga a que cualquier recurso que creemos tenga, como mínimo, el método desplegar() para poder pasarlo a producción.
- Servidor (Objeto Principal / Clase Madre): Define un servidor básico con sus atributos principales (IP, RAM y si está activo). Cumple con todos los requisitos del enunciado: tiene sus constructores, getters, setters y las 3 funciones obligatorias (una con parámetros de entrada, otra que devuelve un valor y otra que lanza excepciones si el servidor está apagado o se intenta entrar como root).
- ServidorWeb (Objeto Heredado / Clase Hija): Hereda todo lo anterior de Servidor y añade 2 funciones nuevas específicas para servidores web: una para asignarle un dominio DNS y otra (marcada como obsoleta) para comprobar el certificado SSL.

2. Generación JavaDoc por consola

El comando exacto que he ejecutado dentro de la carpeta del proyecto ha sido:

javadoc -d doc -author -version *.java

- -d doc: Le dice a Git Bash que cree una carpeta llamada doc y guarde ahí todas las páginas web generadas de la documentación.
- -author y -version: Son las banderas obligatorias para que el compilador lea mis etiquetas @author y @version y las pinte en el HTML, ya que si no se ponen, JavaDoc las ignora por defecto.

PROBLEMS	1	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
----------	---	--------	---------------	----------	-------

```
maken@Makelaptop MINGW64 ~/Documents/GitHub/Practica3-JavaDoc (main)
$ javadoc -d doc -author -version *.java
Loading source file Servidor.java...
Loading source file ServidorWeb.java...
Constructing Javadoc information...
Creating destination directory: "doc\"
Building index for all the packages and classes...
Standard Doclet version 25.0.3+9-LTS
Building tree for all the packages and classes...
Generating doc\Desplegable.html...
Generating doc\Servidor.html...
Servidor.java:27: warning: no @param for ip
    public Servidor(String ip, int ram) {
                   ^
Servidor.java:35: warning: no main description
    /** @return La dirección IP actual del servidor. */
        ^
Servidor.java:37: warning: no main description
    /** @param ip Nueva dirección IP a asignar. */
        ^
Servidor.java:39: warning: no main description
    /** @return true si el servidor está encendido. */
        ^
Servidor.java:49: warning: no @param for puerto
    public void escanearPuerto(int puerto) {
                   ^
Servidor.java:58: warning: no @param for precioPorGB
    public double calcularCosteMensual(double precioPorGB) {
                   ^
Servidor.java:68: warning: no @param for usuario
    public void conexionSSH(String usuario) throws IllegalArgumentException, IllegalStateException {
                   ^
Generating doc\ServidorWeb.html...
ServidorWeb.java:18: warning: no @param for ip
    public ServidorWeb(String ip, int ram, String tecnologiaWeb) {
                   ^
ServidorWeb.java:29: warning: no @param for dominio
    public void asignarDominio(String dominio) {
                   ^
Generating doc\package-summary.html...
Generating doc\package-tree.html...
Generating doc\overview-tree.html...
Generating doc\deprecated-list.html...
Generating doc\allclasses-index.html...
Building index for all classes...
Generating doc\allpackages-index.html...
Generating doc\index-all.html...
Generating doc\search.html...
Generating doc\index.html...
Generating doc\help-doc.html...
9 warnings
```

3. Análisis de las páginas web generadas

Tras lanzar el comando, he entrado en la carpeta doc, he abierto el archivo index.html en el navegador y este es el resultado detallado de la documentación:

A. Página Principal (index.html)

Muestra el índice global del proyecto. Al haber utilizado enlaces de tipo {@link ...} en los comentarios de mi código, la herramienta ha creado hipervínculos automáticos entre la interfaz y las clases, permitiendo navegar por la API de forma súper cómoda.



B. Servidor.java

En la página de este objeto se puede ver cómo se organiza la tabla de métodos y constructores. He documentado los tres métodos exigidos usando los tags correctos para que queden profesionales:

- Método con parámetros (escanearPuerto): He usado el tag `@param` para detallar qué número de puerto hay que pasarle.
- Método que devuelve valor (calcularCosteMensual): He usado el tag `@return` para explicar que devuelve el precio total en euros en formato double.
- Método que lanza excepción (conexionSSH): He usado el tag `@throws` para documentar que el método revienta con un `IllegalStateException` si el servidor está apagado o con un `IllegalArgumentException` si intentan colarse como "root" por seguridad. En la web se ve perfectamente estructurado en una tabla.

```
J Servidor.java > Servidor
1  /**
2   * Clase que representa un Servidor genérico en la nube.
3   * Implementa la interfaz {@link Desplegable}.
4   * * @author Makenzo, Simeón
5   * * @see Desplegable
6   */
7  public class Servidor implements Desplegable {
8
9      private String ip;
10     private int ram;
11     private boolean activo;
12
13     /**
14      * Constructor por defecto que inicializa un servidor básico.
15      */
16     public Servidor() {
17         this.ip = "127.0.0.1";
18         this.ram = 4;
19         this.activo = false;
20     }
21
22     /**
23      * Constructor parametrizado para crear un servidor a medida.
24      * * @param ip Dirección IP asignada al servidor.
25      * * @param ram Cantidad de memoria RAM en Gigabytes (GB).
26      */
27     public Servidor(String ip, int ram) {
28         this.ip = ip;
29         this.ram = ram;
30         this.activo = false;
31     }
32
33     // --- GETTERS Y SETTERS ---
34
35     /** @return La dirección IP actual del servidor. */
36     public String getIp() { return ip; }
37     /** @param ip Nueva dirección IP a asignar. */
38     public void setIp(String ip) { this.ip = ip; }
39     /** @return true si el servidor está encendido. */
40     public boolean isActive() { return activo; }
```

```

42 // --- MÉTODOS REQUERIDOS ---
43
44 /**
45  * <b>Función 1 (Parámetros):</b> Escanea un puerto específico del servidor para comprobar su disponibilidad.
46  * * @param puerto Número del puerto a escanear (ej. 80, 443).
47  * @since 1.5
48  */
49 public void escanearPuerto(int puerto) {
50     System.out.println("Escanear puerto " + puerto + " en la IP " + this.ip);
51 }
52
53 /**
54  * <b>Función 2 (Devuelve valor):</b> Calcula el coste mensual estimado del servidor en base a su RAM.
55  * * @param precioPorGB El coste en euros de 1 GB de RAM por hora.
56  * @return El coste total estimado en euros para un mes (730 horas).
57  */
58 public double calcularCosteMensual(double precioPorGB) {
59     return this.ram * precioPorGB * 730;
60 }
61
62 /**
63  * <b>Función 3 (Lanza Excepción):</b> Intenta conectar al servidor mediante SSH.
64  * * @param usuario Nombre del usuario administrador.
65  * @throws IllegalArgumentException Si el usuario es "root" por motivos de seguridad.
66  * @throws IllegalStateException Si el servidor está apagado.
67  */
68 public void conexionSSH(String usuario) throws IllegalArgumentException, IllegalStateException {
69     if (!this.activo) {
70         throw new IllegalStateException(s: "Error: No se puede conectar. El servidor está APAGADO.");
71     }
72     if ("root".equalsIgnoreCase(usuario)) {
73         throw new IllegalArgumentException(s: "Seguridad denegada: El acceso directo como 'root' está prohibido.");
74     }
75     System.out.println("Conexión SSH establecida con éxito para el usuario: " + usuario);
76 }
77
78 /**
79  * Implementación del método de la interfaz {@link Desplegable}.
80  */
81 @Override
82 public boolean desplegar() {
83     this.activo = true;
84     System.out.println(x: "Servidor desplegado y activo.");
85     return true;
86 }
87 }

```

C. ServidorWeb.java

En esta página, JavaDoc indica arriba del todo que esta clase hereda de la clase madre (extends Servidor).

Además, he querido meterle más complejidad usando el tag `@deprecated` en el método `comprobarSSL()`. En la captura se puede apreciar cómo el navegador tacha automáticamente el nombre del método y genera un cuadro de advertencia gris indicando al usuario que ese método ya no se debe usar porque está anticuado, recomendando usar alternativas modernas.

```
J ServidorWeb.java > ServidorWeb > ServidorWeb(String, int, String)
1  /**
2   * Clase especializada que representa un Servidor Web (ej. Nginx o Apache).
3   * Hereda las propiedades de la clase madre {@link Servidor}.
4   * * @author Makenzo, Simeón
5   * * @version 1.1
6   * * @see Servidor
7   */
8  public class ServidorWeb extends Servidor {
9
10     private String tecnologiaWeb; // Ej: "Nginx", "Apache"
11
12     /**
13      * Constructor para el servidor web especializado.
14      * * @param ip Dirección IP.
15      * * @param ram RAM en GB.
16      * * @param tecnologiaWeb Software de servidor web a utilizar.
17      */
18     public ServidorWeb(String ip, int ram, String tecnologiaWeb) {
19         super(ip, ram);
20         this.tecnologiaWeb = tecnologiaWeb;
21     }
22
23     // --- NUEVOS MÉTODOS ---
24
25     /**
26      * <b>Función Extra 1:</b> Asigna un dominio DNS a este servidor web.
27      * * @param dominio Nombre del dominio (ej: www.miweb.com).
28      */
29     public void asignarDominio(String dominio) {
30         System.out.println("Redirigiendo el dominio " + dominio + " hacia la IP " + getIp());
31     }
32
33     /**
34      * <b>Función Extra 2:</b> Comprueba si el servidor está preparado para servir tráfico HTTPS seguro.
35      * * @deprecated Este método está obsoleto. Se recomienda usar una herramienta externa de certificación.
36      * * @return Siempre devuelve true en este simulador de seguridad.
37      */
38     @Deprecated
39     public boolean comprobarSSL() {
40         return true;
41     }
42 }
```